

AI STUDENT SUCCESS PLATFORM

Production Frontend Master Prompt + API Integration Guide

SIH26 - Ready for Bolt / v0 / Cursor

Production Backend	https://dropout-predictor-2pme.onrender.com
API Base	https://dropout-predictor-2pme.onrender.com/api/v1
Swagger / OpenAPI	https://dropout-predictor-2pme.onrender.com/docs

Core rule: the frontend is a client. FastAPI remains the source of truth for authentication, ML, SHAP, BKT, Gemini, LangGraph, tutoring, sessions, and interventions.

Document Map

1. Production endpoint configuration
2. Non-negotiable architecture rules
3. Modern minimalist visual direction
4. Global application shell and role navigation
5. Authentication and Axios integration
6. Student experience
7. Tutor and adaptive learning sessions
8. Faculty analytics and EWS
9. Intervention workflow and Admin
10. Charts and graph specifications
11. API module structure and error handling
12. Implementation sequence
13. Copy-ready master prompt for AI frontend tools

1. Production Endpoint Configuration

Replace every localhost backend reference with the deployed Render service. Keep the URL in one environment variable and never scatter literal URLs across React components.

```
VITE_API_URL=https://dropout-predictor-2pme.onrender.com/api/v1
```

Recommended Vite environment file:

```
# .env
VITE_API_URL=https://dropout-predictor-2pme.onrender.com/api/v1
```

Do not place backend-only secrets in the frontend. The following must remain on Render/backend only: DATABASE_URL, GOOGLE_API_KEY, JWT_SECRET_KEY, database passwords, and any private service credentials.

2. Architecture Rules the Frontend Must Respect

```
Student signals -> XGBoost -> SHAP -> Risk tier -> Dropout LangGraph -> Intervention -> Faculty

Student learning -> Concept graph -> Adaptive question -> Gemini evaluator -> Tutor LangGraph
-> Gemini Socratic response -> BKT update -> PostgreSQL -> next question/session

JWT -> role check -> ownership check -> API service -> PostgreSQL
```

- Do not calculate risk scores, mastery probabilities, correctness, diagnoses, SHAP contributions, next questions, or intervention decisions in React.
- Do not recreate backend RBAC as if frontend route guards were security. FastAPI 401/403 responses remain authoritative.
- Do not expose reference/expected answers from the tutor question bank.

- Use backend-returned session state and `response.next.question` rather than inventing local session logic.
- Use supportive student-facing language. Do not sensationalize dropout probability.
- Describe SHAP values as predictive risk factors or predictive drivers, never as proven causes.

3. Modern Minimalist Visual Direction

Target a premium, calm SaaS analytics product rather than a generic hackathon template.

Use	Avoid
Generous whitespace, compact cards, clear hierarchy	Excessive gradients, neon colors, glassmorphism
Subtle 1 px borders and soft shadows	Huge floating cards and decorative clutter
Modern typography and restrained accent color	Too many fonts, colors, badges, or visual effects
Responsive charts, readable tables, polished empty states	Random sample chart data presented as real
Skeleton loaders and lightweight motion	Large spinners on every screen
Rounded 12-16 px cards with consistent spacing	Inconsistent radii and spacing

Suggested design tokens

- Page background: very light neutral/slate tone.
- Primary surfaces: white or near-white cards; dark-mode surfaces can be added if already supported.
- Accent: restrained indigo/blue for active navigation and primary actions.
- Risk/status colors: semantic only - green/amber/red for LOW/MODERATE/CRITICAL and success/warning/error.
- Typography: one sans-serif family, strong numeric hierarchy for KPI values, smaller muted metadata text.
- Motion: 150-250 ms transitions for hover, collapse, progress changes, and chart entry; no distracting page choreography.

4. Global Application Shell

Area	Desktop	Mobile
Sidebar	Collapsible left navigation with icons + labels	Drawer / sheet navigation
Header	Page title, optional breadcrumb, notifications, profile, role badge, logout	Compact header with menu trigger and profile
Content	12-column responsive grid, max content width, consistent page padding	Single-column stacking and scroll-safe tables

Feedback	Toast, inline validation, error/empty states	Same patterns with touch-friendly actions
----------	--	---

Role navigation

Role	Navigation
STUDENT	Dashboard, Learn, Sessions, Mastery, History/Profile
FACULTY	Overview, At-Risk Students, Learning Insights, Student Detail, Interventions, Risk History
ADMIN	User Management, System Overview / Faculty views

5. Authentication + Axios Integration

Login uses POST /auth/login. Persist the returned access token in localStorage under access_token. Restore user state on app startup with GET /auth/me.

Step	Behavior
Login	POST /auth/login with email + password
Persist	Save access_token and minimal user state
Restore	GET /auth/me after refresh/app startup
Protect	Attach Authorization: Bearer <token> to protected calls
401	Clear token/user and redirect to /login
403	Show polished unauthorized state; do not retry forever
Role redirect	STUDENT -> /student, FACULTY -> /faculty, ADMIN -> /admin

Shared Axios client

```
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  headers: { "Content-Type": "application/json" },
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("access_token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (response) => response,
```

```

(error) => {
  if (error.response?.status === 401) {
    localStorage.removeItem("access_token");
    localStorage.removeItem("user");
    window.location.href = "/login";
  }
  return Promise.reject(error);
}
);
export default api;

```

6. Student Experience

Student Dashboard

Primary endpoint: GET /student-dashboard/me. Prefer /me routes for logged-in students; do not hardcode or allow changing student_code to inspect someone else.

- Support banner using ON_TRACK, EXTRA_SUPPORT, HIGH_SUPPORT, or NOT_EVALUATED.
- KPI cards: Average Mastery, Mastered Concepts, Concepts Needing Practice, Completed Sessions.
- Recommended-next concept cards with topic, mastery, reason, and action to learn.
- Recent learning sessions with answered/target, correct count, status, starting/final mastery.
- Recent tutor activity with diagnosis and mastery before/after.
- Optional compact mastery overview visualization, but do not calculate mastery locally.

Student Mastery

- GET /student-dashboard/me/mastery for concept cards/progress bars.
- GET /student-dashboard/me/mastery-trend?concept_id=... for a Recharts line chart.
- Show prerequisite concept, mastered state, attempts, and consecutive correct answers.
- Y-axis for mastery trend: 0-100%; X-axis: attempts or timestamps returned by the API.

7. Adaptive Tutor + Learning Sessions

Tutor page

- GET /tutor/concepts to render the concept graph/picker.
- GET /tutor/next-question/{student_code}/{concept_id} for adaptive question selection.
- POST /tutor/adaptive-attempt with student_code, question_id, student_answer, language_code.
- Never send is_correct or expected_answer.
- Display backend evaluation, diagnosis badge, Socratic message, follow-up question, encouragement, prerequisite recommendation, and mastery change.

Preferred final session workflow

Endpoint	Purpose
POST /tutor/sessions/start	Start an adaptive session with concept, target question count, and language
GET /tutor/sessions/student/{student_code}	Session history

GET /tutor/sessions/{session_id}	Session summary/details
GET /tutor/sessions/{session_id}/next-question	Current/next question
POST /tutor/sessions/{session_id}/answer	Submit answer and receive feedback + next state
POST /tutor/sessions/{session_id}/end	Manually end the session

- Start state: concept picker, target questions 1-10, language selector (en-IN, hi-IN, mr-IN).
- Active state: answered/target, accuracy, mastery, active concept, difficulty badge, question, answer textarea.
- Feedback state: diagnosis, Socratic guidance, follow-up question, encouragement, mastery delta.
- Prerequisite state: if active concept changes, show a clear message such as “Revisiting Trees before continuing with Binary Search Trees.”
- Complete state: accuracy, mastery improvement, diagnosis breakdown, attempt timeline, start-another-session action.

8. Faculty Analytics + EWS

Faculty Overview

- GET /faculty/learning/overview for learning KPIs.
- GET /dashboard/summary for risk and intervention KPIs.
- KPI cards: Total Students, Evaluated Students, Average Mastery, Students Needing Practice, Pending Interventions, Socratic Interactions.
- Risk distribution donut chart: LOW, MODERATE, CRITICAL.
- Weak-concepts horizontal bar chart from /faculty/learning/weak-concepts.
- Misconception frequency bar chart from /faculty/learning/misconceptions.

At-Risk Students

- Use /faculty/learning/at-risk-students and/or /dashboard/at-risk.
- Searchable/filterable table with student, code, program, risk tier, risk %, average mastery, top predictive factor, evaluated time, action.
- Filters for LOW/MODERATE/CRITICAL where useful; default faculty attention can emphasize MODERATE/CRITICAL.
- Clicking a student opens the combined faculty student detail view.

Faculty Student Detail

- Use /faculty/learning/student/{student_code} for combined learning + intervention context.
- Use /dashboard/student/{student_code} for EWS detail and selected operational features.
- Use /dashboard/student/{student_code}/risk-history for risk trend chart.
- Show risk tier, risk %, model version, evaluated time, mastery by concept, recent sessions, intervention status.
- Render top SHAP factors as a horizontal contribution chart labelled “Top Predictive Risk Factors”.

9. Intervention Workflow + Admin

Interventions

- GET /dashboard/interventions?status=PENDING_REVIEW for the faculty queue.
- Display student, risk tier, top predictive factors, routed domain, remediation plan, outreach draft, status, mentor notes.
- Statuses: PENDING_REVIEW, APPROVED, REJECTED.
- Before wiring Approve/Reject buttons, inspect deployed Swagger for the exact route and payload; do not invent endpoints.

Admin

Endpoint	UI
GET /auth/users	User table + search + role/status filters
POST /auth/users	Create-user modal; STUDENT requires student_code
PATCH /auth/users/{user_id}/status	Activate/deactivate toggle

The public login UI must never allow users to self-select a role. Role creation/management remains an ADMIN operation.

10. Required Charts and Graphs

Screen	Chart	Data source	Notes
Student Mastery	Line chart - mastery trend	/student-dashboard/me/mastery-trend	Attempt/time on X, mastery % on Y
Student Dashboard	Concept mastery progress	/student-dashboard/me or /mastery	Progress bars/rings; concise, not decorative
Faculty Overview	Donut - risk distribution	/dashboard/summary	LOW/MODERATE/CRITICAL counts
Learning Insights	Horizontal bar - weak concepts	/faculty/learning/weak-concepts	Concept vs average mastery / below-mastery count
Learning Insights	Bar - misconceptions	/faculty/learning/misconceptions	Diagnosis/misconception frequency
Faculty Student Detail	Horizontal bar - SHAP factors	/dashboard/student/{code}	Label as predictive factors, not causes
Faculty Student Detail	Line - risk history	/dashboard/student/{code}/risk-history	Risk % over snapshots/evaluations
Faculty Student Detail	Mastery by concept	/faculty/learning/student/	Bars/progress indicators by

		{code}	concept
--	--	--------	---------

All charts must use actual API responses. Provide ResponsiveContainer, readable labels, tooltips, loading skeletons, and meaningful empty states. Do not invent random production-looking data.

11. Frontend API Module Structure

```
src/
  api/
    client.ts
    authApi.ts
    studentApi.ts
    tutorApi.ts
    facultyApi.ts
    ewsApi.ts
    adminApi.ts
  components/
  layouts/
  pages/
  hooks/
  utils/
  types/
```

- Keep endpoints and request logic in API modules, not scattered across page components.
- Use reusable hooks for loading/error/empty request states if the project is large enough.
- Use frontend route guards only for UX; always honor backend 401/403.
- Handle null values and empty arrays safely. Format ISO timestamps locally.
- For 422, surface field validation messages. For 503, show AI service temporarily unavailable without corrupting session state.

12. Page-to-Endpoint Mapping

Page	Primary endpoints	UI focus
Login	POST /auth/login; GET /auth/me	Email/password, auth restore, role redirect
Student Home	GET /student-dashboard/me	Support, KPIs, recommendations, sessions, activity
Student Mastery	GET /student-dashboard/me/mastery; mastery-trend	Concept mastery + trend chart
Tutor / Learn	GET /tutor/concepts; next-question; adaptive-attempt	Question + Socratic feedback
Learning Session	POST sessions/start; POST {id}/answer; GET {id}	Progress, adaptive questions, feedback, summary
Tutor History	GET /tutor/history/{own student_code}	Timeline/filter by concept
Faculty Overview	GET /faculty/learning/overview;	Risk + learning KPIs + charts

	/dashboard/summary	
At-Risk Students	GET /faculty/learning/at-risk-students; /dashboard/at-risk	Search/filter table
Learning Insights	GET weak-concepts; misconceptions	Bar charts + rankings
Faculty Student Detail	GET /faculty/learning/student/{code}; /dashboard/student/{code}; risk-history	Risk + SHAP + mastery + sessions + intervention
Interventions	GET /dashboard/interventions?status=...	Queue + review actions
Admin Users	GET/POST /auth/users; PATCH /auth/users/{id}/status	User management

13. Implementation Sequence

1. Inspect the existing Bolt frontend. Do not rewrite the whole app immediately.
2. Add VITE_API_URL with the production Render API base.
3. Create shared Axios client and response handling.
4. Connect login and /auth/me.
5. Add role-aware routes and shells.
6. Connect Student Dashboard.
7. Connect Student Mastery and mastery trend chart.
8. Connect Tutor.
9. Connect Learning Sessions and session summary.
10. Connect Faculty Overview and charts.
11. Connect At-Risk Students and Faculty Student Detail.
12. Connect Learning Insights and Interventions.
13. Connect Admin User Management.
14. Add skeletons, empty/error states, responsive polish, accessibility labels.
15. Run npm build and production end-to-end tests against the deployed backend.

14. Copy-Ready Master Prompt for Bolt / v0 / Cursor

Build/refine a production-style responsive frontend for our AI Student Success Platform using React + Vite + Tailwind CSS, Axios, React Router, Recharts, and Lucide icons.

PRODUCTION BACKEND:
<https://dropout-predictor-2pme.onrender.com>

API BASE:

<https://dropout-predictor-2pme.onrender.com/api/v1>

SWAGGER:

<https://dropout-predictor-2pme.onrender.com/docs>

FRONTEND ENV:

VITE_API_URL=<https://dropout-predictor-2pme.onrender.com/api/v1>

The backend is already implemented and is the source of truth. Do NOT recreate ML, XGBoost scoring, SHAP explanations, BKT calculations, Gemini evaluation, LangGraph routing, adaptive question selection, risk logic, session state, intervention decisions, or authorization rules in React.

DESIGN:

Make the UI modern, minimalist, premium, and SaaS-like. Use generous whitespace, subtle borders, soft shadows, rounded cards, restrained accent colors, modern typography, clean tables, compact badges, responsive layouts, skeleton loaders, polished empty states, and subtle transitions. Avoid excessive gradients, neon, glassmorphism, giant cards, decorative clutter, and random chart data.

GLOBAL SHELL:

Create role-specific shells with a collapsible desktop sidebar and mobile drawer. Add a compact header with page title, optional breadcrumb, notifications, user avatar/name, role badge, and logout.

AUTH:

POST /auth/login. Store access_token in localStorage. Attach Authorization: Bearer <token> using one shared Axios client. Restore user with GET /auth/me. On 401 clear auth and redirect /login. On 403 show unauthorized. Route STUDENT -> /student, FACULTY -> /faculty, ADMIN -> /admin.

STUDENT:

Dashboard from GET /student-dashboard/me. Show supportive status, KPIs (average mastery, mastered concepts, concepts needing practice, completed sessions), recommended concepts, recent sessions, and tutor activity. Do not prominently show raw dropout probability to students.

MASTERY:

Use /student-dashboard/me/mastery and /student-dashboard/me/mastery-trend. Add concept mastery cards/progress bars and a Recharts line chart for mastery progression.

TUTOR:

Use /tutor/concepts, /tutor/next-question/{student_code}/{concept_id}, and POST /tutor/adaptive-attempt. Render a focused question card, natural-language answer field, difficulty badge, evaluation/diagnosis, Socratic message, follow-up question, encouragement, prerequisite recommendation, and mastery change. Never send is_correct or expected_answer.

LEARNING SESSIONS:

Prefer POST /tutor/sessions/start, GET /tutor/sessions/{session_id}, GET /next-question, POST /answer, POST /end. Show answered/target, accuracy, current mastery, active concept, question, feedback, prerequisite transitions, and a final summary with mastery improvement and diagnosis breakdown. Always render backend-returned response.next.question.

FACULTY:

Use /faculty/learning/overview and /dashboard/summary for KPI cards. Add a risk distribution donut chart, weak-concepts horizontal bar chart, misconception frequency bar chart, at-risk student table, and pending intervention summary.

FACULTY STUDENT DETAIL:

Combine /faculty/learning/student/{student_code}, /dashboard/student/{student_code}, and risk-history. Show risk tier, risk %, top predictive SHAP factors as horizontal contribution bars, mastery by concept, risk-history line chart, recent sessions, and intervention status. Label SHAP values as Top Predictive Risk Factors, not causes.

INTERVENTIONS:

Use /dashboard/interventions?status=PENDING_REVIEW. Show student, risk tier, predictive factors, routed domain, remediation plan, outreach draft, status, mentor notes. Before wiring Approve/Reject, inspect Swagger for exact action routes/payloads; never invent endpoints.

ADMIN:

Use GET/POST /auth/users and PATCH /auth/users/{user_id}/status. Build a clean user table, search, role/status filters, create-user modal, and activate/deactivate controls. The login page must not let users select their own role.

API ORGANIZATION:

Create src/api/client.ts, authApi.ts, studentApi.ts, tutorApi.ts, facultyApi.ts, ewsApi.ts, adminApi.ts. Do not scatter literal URLs across components. Handle 401, 403, 404, 422, 500, 503 consistently.

WORKFLOW:

First inspect the existing Bolt frontend and report pages, routes, components, hardcoded/mock data, auth implementation, API calls, unused Supabase/Firebase/Bolt dependencies, and files that need modification. Then implement incrementally: Axios -> Login -> /auth/me -> role guards -> Student Dashboard -> Mastery -> Tutor -> Sessions -> Faculty Overview -> At-Risk -> Student Detail -> Learning Insights -> Interventions -> Admin. Test each stage before proceeding. Preserve useful existing UI while upgrading inconsistent sections to the modern minimalist direction above.

15. Final QA Checklist

- No localhost API URLs remain in production code except optional development env files.
- No demo passwords, JWTs, API keys, DB URLs, or secrets are committed to the frontend repository.
- Login, refresh auth restore, logout, 401, and 403 are tested.
- STUDENT cannot inspect another student by changing a URL or student_code.
- FACULTY analytics load with real backend data.
- Tutor submits natural-language answers without expected_answer/is_correct.
- Learning sessions use backend-returned next-question state.
- Charts use real API data and have loading/empty/error states.
- SHAP is labelled predictive, not causal.
- Responsive behavior is checked on desktop, tablet, and mobile widths.
- npm run build completes successfully.
- Production frontend is tested end-to-end against <https://dropout-predictor-2pme.onrender.com>.

End of Frontend Production Handoff